A closer look at a simple experiment

Our team investigated how the resistance of a component affects the current in a small electrical circuit. We connected a resistor to a variable power supply and used a multimeter to measure the voltage across it. Each reading was written in a notebook before the voltage was changed. We repeated the measurements to check whether the pattern was consistent.

The current increased as the applied voltage increased. For the range of values tested, the relationship was approximately linear. This observation agrees with the expectation for an ordinary resistor operating at a steady temperature. Small differences between repeated readings may be explained by instrument resolution, loose contacts, or changes in the temperature of the component.

We also wrote a short program to summarize the measurements. The program keeps valid numerical readings, calculates their average, and reports the difference between the highest and lowest values. This helped us identify a reading that had been entered incorrectly. We checked the original notebook before changing that value, so the summary remained traceable to the measurements.

The experiment illustrates why a result should be examined alongside the evidence that produced it. A summary value can help a reader find a pattern, but it cannot explain every detail of a measurement. The next step would be to repeat the procedure with additional components and a wider range of temperatures. More observations would help us distinguish a consistent effect from a one-off variation.

The source behind the summary

```
def summarize_readings(readings):
    # Retain numeric observations in their original order.
    valid = []
    for reading in readings:
        if isinstance(reading, (int, float)):
            valid.append(reading)
    if not valid:
        return {"count": 0, "average": None}
    total = sum(valid)
    average = total / len(valid)
    return {"count": len(valid), "average": average}


def measurement_spread(readings):
    valid = [value for value in readings
             if isinstance(value, (int, float))]
    if len(valid) < 2:
        return 0
    smallest = min(valid)
    largest = max(valid)
    return largest - smallest


def compare_trials(first_trial, second_trial):
    first = summarize_readings(first_trial)
    second = summarize_readings(second_trial)
    if not first["count"] or not second["count"]:
        return None
    return second["average"] - first["average"]
```